

CHƯƠNG 19: HỌC TỪ CÁC VÍ DỤ

TRÍ TUỆ NHÂN TẠO - ARTIFICIAL INTELLIGENCE

Nội dung Chương 19

- ◇ 19.1 Các Hình thức Học tập (Forms of Learning)
- ◇ 19.2 Học có Giám sát (Supervised Learning)
- ◇ 19.3 Học từ các Cây Quyết định (Learning Decision Trees)
- ◇ 19.4 Lựa chọn Mô hình (Model Selection) & Hàm Suy hao (Loss)
- ◇ 19.6 Hồi quy Tuyến tính & Phân loại (Linear Regression)
- ◇ 19.7 Máy Vector Hỗ trợ (SVM) & K-NN
- ◇ 19.8 Học Cụm tổng hợp (Ensemble Learning)

19.1 Các Hình thức Học tập (Forms of Learning)

◇ Một Tác nhân AI được gọi là **Học tập (Learning)** nếu nó có thể cải thiện hiệu suất hoạt động thông qua các trải nghiệm tương lai.

◇ **3 Phân nhánh chính của Học máy (Machine Learning):**

1. **Học không giám sát (Unsupervised Learning):** Học sinh tự tìm ra quy luật từ dữ liệu mà không có đáp án mẫu. (Ví dụ: Gom cụm dữ liệu).
2. **Học tăng cường (Reinforcement Learning):** Học sinh vừa làm vừa quan sát phần thưởng/hình phạt.
3. **Học có giám sát (Supervised Learning):** Học sinh học dựa trên bộ dữ liệu đã được gán nhãn đáp án chuẩn bởi con người.

19.2 Học có Giám sát (Supervised Learning)

◇ Định nghĩa bài toán:

- Cho trước một tập huấn luyện gồm các cặp $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$.
- Trong đó x_i là đầu vào (đặc trưng), y_i là đầu ra (nhãn).
- **Mục tiêu:** Tìm một hàm **giả thuyết (hypothesis)** h sao cho $h(x) \approx y$ với mọi điểm dữ liệu, cả dữ liệu chưa từng thấy (Tổng quát hóa).

◇ Phân loại:

- **Phân loại (Classification):** Đầu ra y là rời rạc (Ví dụ: Bệnh/Không Bệnh, Chó/Mèo).
- **Hồi quy (Regression):** Đầu ra y là liên tục (Ví dụ: Dự đoán giá nhà).

19.3 Học từ Cây Quyết định (Decision Trees)

◇ **Cây Quyết định (Decision Tree):** Là một trong những thuật toán học máy lâu đời và dễ hiểu nhất.

- Hoạt động bằng cách chia nhỏ dữ liệu liên tiếp thành các nhánh dựa trên một chuỗi các câu hỏi *IF-THEN*.
- **Ví dụ:** Có nên chờ bàn ở nhà hàng này không? (Xét các thuộc tính: Số khách, Thời gian chờ, Cuối tuần, Có đôi không).

Example	Input Attributes										Output
	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	WillWait
x ₁	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	y ₁ = Yes
x ₂	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	y ₂ = No
x ₃	No	Yes	No	No	Some	\$	No	No	Burger	0-10	y ₃ = Yes
x ₄	Yes	No	Yes	Yes	Full	\$	Yes	No	Thai	10-30	y ₄ = Yes
x ₅	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	y ₅ = No
x ₆	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0-10	y ₆ = Yes
x ₇	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	y ₇ = No
x ₈	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0-10	y ₈ = Yes
x ₉	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	y ₉ = No
x ₁₀	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	y ₁₀ = No
x ₁₁	No	No	No	No	None	\$	No	No	Thai	0-10	y ₁₁ = No
x ₁₂	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	y ₁₂ = Yes

19.3 Chọn Thuộc tính (Entropy & Info Gain)

◇ Để xây dựng cây nhỏ nhất (hiệu quả nhất), ta phải chọn được **Thuộc tính quan trọng nhất** để đưa lên làm Nút gốc (Root).

◇ **Đo lường độ hỗn loạn (Entropy):**

- $H(V) = - \sum_k P(v_k) \log_2 P(v_k)$
- Tập dữ liệu càng tinh khiết (toàn YES hoặc toàn NO), Entropy càng tiến về 0. Càng lẫn lộn (50-50), Entropy càng tiến về 1.

◇ **Mức tăng thông tin (Information Gain - IG):**

- Chọn thuộc tính làm giảm đi độ hỗn loạn lớn nhất.
- $Gain(A) = Entropy(\text{Tập ban đầu}) - Entropy(\text{Sau khi chia theo } A)$

19.3 Overfitting và Cắt tỉa (Pruning)

◇ Khớp quá mức (Overfitting):

- Nếu ta cứ chia nhánh cho đến khi mọi chiếc lá đều đạt $\text{Entropy} = 0$, cây sẽ **học vẹt** mọi nhiễu (noise) của tập huấn luyện.
- Cây sẽ hoạt động rất tệ trên dữ liệu thực tế mới.

◇ Cắt tỉa (Pruning):

- Giải pháp để chống Overfitting. Cắt bỏ bớt những nhánh sâu không mang lại thông tin thực sự (sử dụng kiểm định thống kê χ^2).
- Giúp mô hình **Tổng quát hóa (Generalize)** tốt hơn.

19.4 Lựa chọn Mô hình & Đánh giá chéo

◇ Tập dữ liệu chia làm 3 phần:

1. **Training Set:** Dùng để tìm thông số mô hình (VD: Xây cây).
2. **Validation Set:** Dùng để điều chỉnh siêu tham số và tránh Overfitting (Chọn độ sâu cho cây).
3. **Test Set:** Để đánh giá khách quan hiệu suất cuối cùng. (Luôn giấu kín).

◇ **K-fold Cross Validation (Đánh giá chéo):**

- Chia dữ liệu thành K phần bằng nhau.
- Lấy 1 phần làm Test, $K - 1$ phần làm Training. Lặp K lần và tính trung bình lỗi để mô hình đáng tin cậy hơn.

19.4 Hàm Suy hao (Loss) & Regularization

◇ Hàm Suy hao (Loss Function - $L(x, y, \hat{y})$):

- Đo lường mức độ "Sai" của giả thuyết h . L càng nhỏ càng tốt.
- Phân loại: Dùng lỗi 0/1 (Đúng/Sai).
- Hồi quy: Dùng Lỗi bình phương $(y - \hat{y})^2$ hoặc Lỗi tuyệt đối $|y - \hat{y}|$.

◇ Chính quy hóa (Regularization):

- Trừng phạt các mô hình quá phức tạp để ép nó phải đơn giản.
- $Cost = Loss(Data, Model) + \lambda \times Complexity(Model)$
- L1 Regularization: Có thể ép tham số về 0. L2 Regularization: Ép tham số nhỏ lại.

19.5 Học PAC (Theory of Learning)

◇ Làm thế nào ta biết chắc chắn thuật toán AI thực sự "Học" được chứ không phải an may?

◇ **Học PAC (Probably Approximately Correct):**

- Có khả năng rất cao (Probably) là mô hình sẽ đúng ở một mức xấp xỉ chấp nhận được (Approximately Correct).
- Khung lý thuyết này cung cấp công thức tính: Cần thu thập *bao nhiêu ví dụ (Sample complexity)* để đảm bảo sai số thuật toán luôn dưới mức ϵ với độ tin cậy $1 - \delta$.

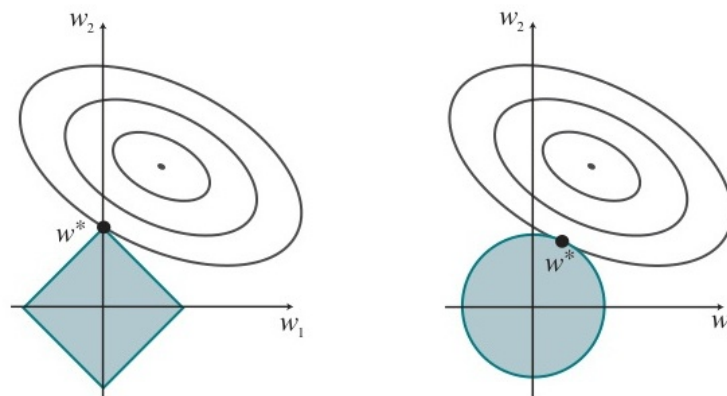
19.6 Hồi quy Tuyến tính (Linear Regression)

◇ **Hồi quy Tuyến tính:** Cố gắng khớp các điểm dữ liệu bằng một đường thẳng (hoặc siêu phẳng trong không gian nhiều chiều).

- Phương trình dự đoán: $h_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$

◇ **Bài toán tối ưu hóa:**

- Ta cần tìm vector trọng số $\mathbf{w}$ làm cực tiểu hóa hàm Loss Bình phương (Mean Squared Error).



19.6 Suy giảm độ dốc (Gradient Descent)

- ◇ Trong nhiều bài toán, không thể dùng Đại số tuyến tính để tìm $\mathbf{w}$ tối ưu.
- ◇ **Gradient Descent (Thuật toán Suy giảm độ dốc):**
 - Là xương sống của mọi thuật toán Học Sâu (Deep Learning) ngày nay.
 - Tưởng tượng một người đứng trên núi có sương mù, muốn tìm đường xuống đáy thung lũng sâu nhất. Người đó sẽ cảm nhận độ dốc dưới chân và bước về hướng dốc đi xuống.

◇ Công thức cập nhật Trọng số:

- $w_j \leftarrow w_j - \alpha \frac{\partial}{\partial w_j} Loss(w)$
- α là **Tốc độ học (Learning Rate)** - quyết định bước chân dài hay ngắn.

19.7 Học máy phi tham số & k-NN

◇ Hồi quy tuyến tính có $\mathbf{w}$ gọi là mô hình *Có tham số (Parametric)*. Khi đã học xong $\mathbf{w}$, ta ném Dữ liệu huấn luyện đi.

◇ **Mô hình Phi tham số (Non-parametric Models):**

- Không có tập tham số cố định. Phải lưu trữ LẠI TOÀN BỘ dữ liệu.

◇ **k-NN (k-Nearest Neighbors):**

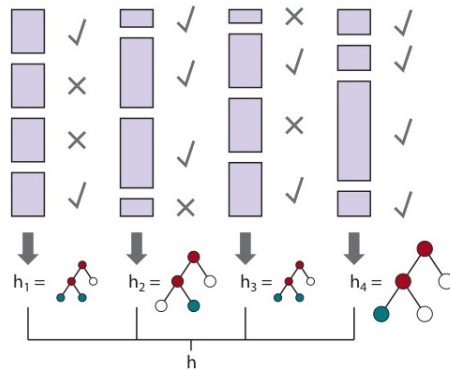
- Để dự đoán một điểm dữ liệu mới x_{new} , hãy tìm k điểm gần nhất với nó trong kho dữ liệu cũ, rồi "Bầu chọn đa số" (Majority vote).
- Rất đơn giản, nhưng gặp phải **Lời nguyền đa chiều (Curse of Dimensionality)**: Trong không gian nghìn chiều, mọi điểm đều cách rất xa nhau.

19.7 Máy vector hỗ trợ (SVM)

◇ **Support Vector Machines (SVM):** Một trong những thuật toán Đỉnh cao nhất trước kỷ nguyên Deep Learning.

◇ **Mục tiêu:** Thay vì vẽ bừa một đường thẳng chia đôi ranh giới phân loại, SVM đi tìm **Siêu phẳng có Lệ cực đại (Maximum Margin Hyperplane)**.

- Đường phân ranh giới nằm xa nhất đối với mọi điểm dữ liệu gần nhất ở cả hai lớp.
- Các điểm dữ liệu nằm ngay sát biên đường Lệ này gọi là ****Vector hỗ trợ (Support Vectors)****.



19.7 Kỹ thuật Hạt nhân (Kernel Trick) trong SVM

◇ **Vấn đề:** Hầu hết dữ liệu trong thực tế đều đan xen phức tạp (Phi tuyến tính), không thể chia cắt bằng 1 đường thẳng.

◇ **Kernel Trick:**

- Bí mật của SVM: Biến dữ liệu từ không gian 2D lên không gian Đa chiều (thậm chí Vô hạn chiều) sao cho ở trên đó, chúng ta có thể *Cắt tuyến tính* bằng một mặt phẳng.
- Phép tính ánh xạ (Mapping) vô cùng nặng, nhưng nhờ Kernel Trick

(Ví dụ: RBF Kernel, Polynomial Kernel), toán học tính Tích vô hướng (Dot Product) ở không gian mới vô cùng nhanh gọn mà không tốn chi phí biến đổi.

19.8 Học Cụm tổng hợp (Ensemble Learning)

- ◇ *"Một cá nhân có thể sai, nhưng đám đông sẽ đúng".*
- ◇ Thay vì chỉ huấn luyện 1 mô hình h , ta sẽ huấn luyện ****hàng trăm**** mô hình $h_1, h_2, \dots, h_k$ và cho chúng bỏ phiếu chung (Voting). Tập hợp này gọi là **Ensemble**.
- ◇ **Bagging (Bootstrap Aggregating)**: Lấy mẫu ngẫu nhiên từ tập dữ liệu lớn để tạo ra các bộ dữ liệu nhỏ. Mỗi mô hình học trên 1 bộ nhỏ, sau đó lấy trung bình. Tránh Overfitting cực tốt.

19.8 Rừng ngẫu nhiên (Random Forests) & Boosting

◇ Rừng ngẫu nhiên (Random Forests):

- Là phiên bản nâng cấp của Bagging chuyên dành cho Cây Quyết Định.
- Chứa hàng nghìn cây. Mỗi cây không chỉ nhận dữ liệu ngẫu nhiên, mà còn bị *ép* chỉ dùng một nhóm thuộc tính ngẫu nhiên khi phân tách nhánh.

◇ Boosting (Ví dụ: AdaBoost, Gradient Boosting):

- Huấn luyện mô hình một cách **Tuần tự**.
- Mô hình thứ $i + 1$ sẽ tập trung "Học lại" và "Chỉnh sửa" những điểm dữ liệu khó mà Mô hình thứ i đoán sai.
- Từ nhiều học sinh kém cỏi (Weak learners), Boosting biến chúng thành một Giáo sư xuất chúng (Strong learner).

Tóm tắt Chương 19

- ◇ **Học Máy (Machine Learning):** Chuyển AI từ kỹ nguyên "Lập trình bằng tay" sang kỹ nguyên "Đưa dữ liệu để tự học".
- ◇ **Cây Quyết Định (Decision Tree):** Lỗi lý thuyết cơ bản dựa trên Entropy và mức tang thông tin.
- ◇ **Loss Function & Gradient Descent:** Hạt nhân toán học phía sau quá trình tự cập nhật trọng số trong suốt chiều dài lịch sử AI.
- ◇ **SVM & Ensemble Learning:** Những "Gã Khổng Lồ" của học máy phi tuyến tính, giúp các hệ thống đạt mức chính xác vượt trội mà không bị khớp quá mức.